

A2A Goes Stable

What changed, why, and what's next

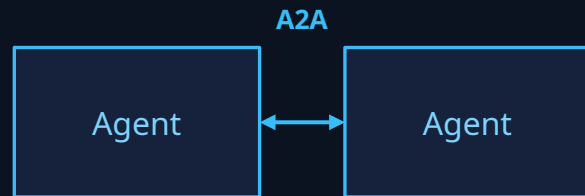
Sam Betts

Engineering Technical Lead · Cisco Systems

Kuba Herczyński

Staff Software Engineer · Google

Both A2A Protocol maintainers · #AGNTCon #MCPCon



Two core maintainers

Sam Betts

Engineering Technical Lead · Cisco Systems

Contributor to the A2A specification and SDKs. Works on AGNTCY. Fourteen years building cloud-native platforms, security and AI systems.

Kuba Herczyński

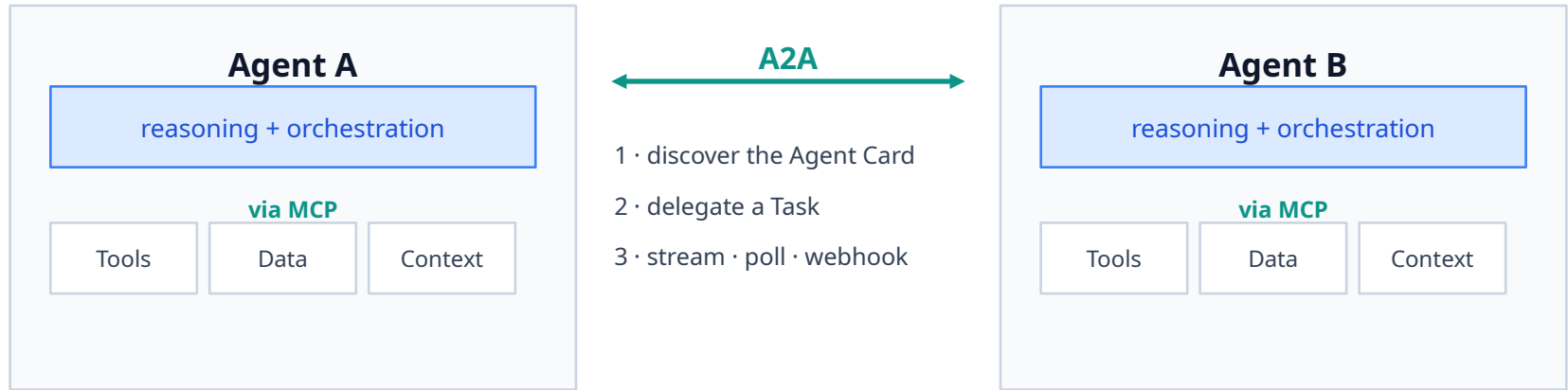
Staff Software Engineer · Google

Contributor to the A2A specification and its adoption. Authored the v0.3 to v1.0 backward-compatibility design.

A2A is a hosted project of the Agentic AI Foundation — the foundation running this conference.

150+ partner organisations. TSC: AWS · Cisco · Google · IBM Research · Microsoft · Salesforce · SAP · ServiceNow

A2A in 60 seconds



MCP is inside an agent. A2A is between agents.

Most production systems run both.

THE OBVIOUS QUESTION

Why not MCP to agent?



Agents adapt, plan and solve problems. Tools reply.

Why v1.0 — and why now

01

Governance

A Linux Foundation project with an eight-company steering committee. Not one vendor's spec.

02

Reach

Six official SDKs shipped: Python, Go, Java, JavaScript, .NET and Rust.

03

Design intent

A hardening release, not a reinvention. Core ideas intact, rough edges removed.

v0.3 was something to experiment with. v1.0 is something to commit to.

What “stable” actually means

01

One normative source of truth

a2a.proto is now the universal definition of the protocol, not a gRPC-specific file.

02

Explicit compatibility rules

A defined versioning strategy, with the protocol version declared per interface.

03

Bindings that must agree

JSON-RPC, HTTP+JSON and gRPC are formally specified with equivalence guarantees.

Stable $\neq$ finished. Stable means you can plan around it.

ACT II

What v1.0 delivers

Signed Agent Cards · Web-aligned architecture · Multi-tenancy · Modern security

Four things v1.0 puts on the table

Signed Agent Cards

Cryptographic verification of identity and metadata — trust established before interaction.

Multi-tenancy

A single endpoint can securely host many agents.

Heterogeneous environments

Multi-protocol bindings and version negotiation.
No lock-in to one stack.

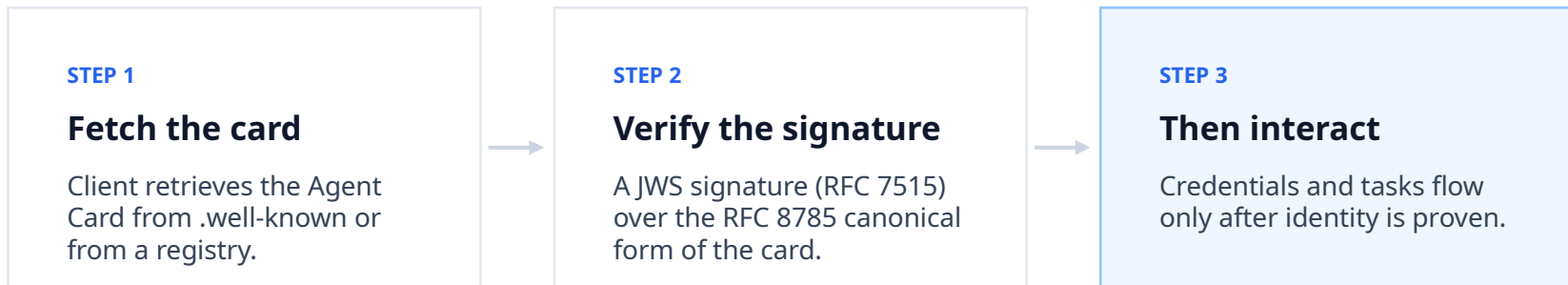
Improved security posture

Modern auth flows in, legacy patterns out.

Highlighted: the two we will look at in detail.

Signed Agent Cards

In v0.3, an Agent Card was effectively trust-on-first-use.



Closes at the discovery layer

lookalike cards in registries · stale cards with swapped auth · compromised discovery paths

Verify before you send credentials, before you send a task — before you send anything.

Web-aligned architecture

WIRE	JSON + HTTP · JSON-RPC · gRPC
DELIVERY	polling · streaming · webhooks
STYLE	stateless · layered · HTTP semantics

**“An A2A interaction
can begin with a
single HTTP
request.”**

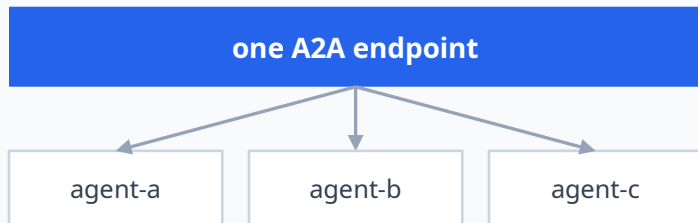
— A2A v1.0 announcement

Your existing load balancers, API gateways, WAFs and tracing already work. You do not need an agent mesh.

Multi-tenancy and modern security

Multi-tenancy

Route by URL sub-path, auth credentials, or an opaque tenant field in the Agent Card.



Improved security posture

- ✓ Device Code flow (RFC 8628)
- ✓ pkce_required on Auth Code (RFC 7636)
- ✗ Implicit and Password flows removed

Aligned with the OAuth 2.0 Security BCP.

Enterprise-ready $\neq$ enterprise-heavy. Same protocol, better defaults.

ACT III

Migration & extensibility

Breaking changes · backward compatibility by design · extensions and bindings

THE HONEST SLIDE

Yes — v0.3 → v1.0 breaks the wire

IMPACT	WHAT CHANGED	WHY
CRITICAL	TextPart / FilePart / DataPart → one unified Part	Member presence replaces the kind discriminator
HIGH	Enum values → SCREAMING_SNAKE_CASE	ProtoJSON compliance: completed → TASK_STATE_COMPLETED
HIGH	Stream events lose kind and final	Wrapper members statusUpdate / artifactUpdate instead
HIGH	Agent Card collapses into supportedInterfaces[]	One interface list; protocolVersion moves inside each
HIGH	Errors now use google.rpc.Status	Replaces RFC 9457; ErrorInfo carries reason and domain

Ordered by the impact rating in the spec's own migration guide. →

Compatibility, baked into the Agent Card

The Agent Card is public — it lives in .well-known files, registries and emails between teams. You cannot ship two versions of it.

PROTOBUF DEFINITION

v1.0 fields only

Clean. No legacy debt carried into the type definitions.

JSON WIRE FORMAT

v1.0 + v0.3 fields

Legacy field names preserved. SDKs translate on marshal and unmarshal.

A v0.3 client can still parse a v1.0 server's card — and the reverse.

Clean core. Compatibility at the edges.

Per-interface versioning

v0.3: the protocol version was a property of the whole Agent Card. One agent, one version.

v1.0: protocol_versions moved into each interface.



Version negotiation becomes part of transport negotiation.

Separate routes for your LB

Separate metrics in tracing

Opt into back-compat per transport

THE PAYOFF

All four combinations work

	v0.3 SERVER	v1.0 SERVER
v0.3 client	✓ Native	✓ Legacy interface + card back-compat
v1.0 client	✓ Card back-compat	✓ Native

Progressive migration. No flag day. No frozen ecosystem.

MEASURED, NOT ASSERTED — SCAN FOR THE LIVE DASHBOARD

Every night the ITK tests 8 SDK/version pairs — v1.0 and v0.3, all three bindings.

The v0.3 cells above use each SDK's compatibility layer: opt-in in JS, not yet in Rust.



A stable core, not a frozen ecosystem

Custom protocol bindings

Replace the default transport while preserving A2A semantics.

Extensions

Add capabilities on top of the core. Advertised on the Agent Card, so clients can negotiate.

A2A core

Stable. Governed. Protected by semver.

Both extension points have governance, so the ecosystem does not fragment.

What the community is building now

NEXT PROTOCOL VERSION

v1.1 — bidirectional streaming, later this year

Backward compatible within v1.x — the semver promise, in practice.

01

SDK parity

All six SDKs support v1.0. Four have cut stable releases; .NET and Rust are pre-stable.

02

Validation tooling

Inspector, the TCK, the ITK — and a new official A2A CLI.

03

Community-led development

Standard processes for contributing to the spec, SDKs and tooling.

Full roadmap: a2a-protocol.org/latest/roadmap

TWO THINGS TO DO

Try it. Then help shape it.

01

Try v1.0 today

a2a-protocol.org

NEW · official A2A CLI

Pick your SDK, or start with the CLI. On v0.3? Migration is progressive, not a rewrite.



02

Join the working group

github.com/a2aproject

Concrete asks right now:
bidirectional streaming for
v1.1, .NET and Rust SDK
parity, custom protocol
bindings.



If you're building an agent — not just a long-running tool — expose it over A2A.

Thank you

Questions?

Sam Betts · Cisco Systems · github.com/tehsmash

Kuba Herczyński · Google · github.com/herczyn

a2a-protocol.org · github.com/a2aproject



a2a-protocol.org

A2A Protocol is a Linux Foundation project. TSC: AWS · Cisco · Google · IBM Research · Microsoft · Salesforce · SAP · ServiceNow